

Lab 7: Polymorphism & Virtual functions

Theory:

- Define polymorphism. Explain its types.
- Why do we need virtual function? Explain pure virtual function in brief.

Lab:

Question: 1

WAP to find the cube of a integer , float and double number using the concept of function overloading(passing single argument to the function).

```
#include<iostream>
using namespace std;

void findcube(int a){
    cout<<"The cube of the integer number is:"<<a*a*a<<endl;
}

void findcube(float b)
{
    cout<<"The cube of the float number is:"<<b*b*b<<endl;
}

void findcube(double c)
{
    cout<<"The cube of the double number is:"<<c*c*c;
}

int main( )
{
    int x;
    float y;
    double z;
    cout<<"Enter values for int, float double type variables
respectively:";
    cin>>x>>y>>z;
    findcube(x);
    findcube(y);
    findcube(z);
```

```
    return 0;
}
```

Output:

Question: 2

WAP in C++ to initialize a pointer and access the value stored in it.

```
#include <iostream>
using namespace std;

int main() {
    int num = 25;           // normal variable
    int *p;                 // pointer declaration

    p = &num;               // pointer initialization with address of num

    cout << "Value of num: " << num << endl;
    cout << "Address of num: " << &num << endl;
    cout << "Pointer p stores (address of num): " << p << endl;
    cout << "Value accessed using pointer: " << *p << endl;

    return 0;
}
```

Output:

Question: 3

Write a C++ program to demonstrate the use of a pointer to base class and a pointer to derived class.

```
#include<iostream>
using namespace std;
class Base
{
    public:
        int m;
        void display( )
        {
            cout<<"The value of m="<<m<<endl;
        }
};
```

```

class Derived:public Base
{
    public:
        int d;
        void display( )
        {
            cout<<"The value of m and d="<<m<<" "<<d<<endl;
        }
};

int main( )
{
    Base *bptr;
    Derived D;
    bptr=&D;
    bptr->m=10;
    //bptr->d=20; error
    bptr->display( );
    Derived *dptr;
    dptr=&D;
    dptr->d=20;
    dptr->display( );
    return 0;
}

```

Output:

Question: 4

Write a program that shows the runtime polymorphism in C++.

```

#include<iostream>
using namespace std;
class Animal
{
    public:
        virtual void display( ) //Virtual function
        {
            cout<<"Animal class:"<<endl;
        }
};

class Cow:public Animal

```

```

{
public:
    void display( )    //virtual function redefined in derived
    {
        cout<<"Cow class:"<<endl;
    }
};

class Dog: public Animal
{
public:
void display( )
{
cout<<"Dog class:"<<endl;
}
};

int main( )
{
Animal *Anm;
Animal a1;
Cow c1;
Dog d1;
Anm=&a1;
Anm->display( );
Anm=&c1;
Anm->display( );
Anm=&d1;
Anm->display( );
return 0;
}

```

Output:

Question: 5

Write a C++ program to demonstrate the concept of abstract class.

```
#include<iostream>
using namespace std;
class polygon //Abstract class
{
protected:
float length,height;
public:
void input( )
{
cout<<"Enter the values to the length and height:"<<endl;
cin>>length>>height;
}
virtual void area( )=0; //pure virtual function
};

class rectangle: public polygon{
public:
void display( )
{
    cout<<"The length of the rectangle="<<length<<endl;
    cout<<"The height of the rectangle="<<height<<endl;
}
void area( )
{
cout<<"The area of the rectangle="<<length*height<<endl;
}
};

class triangle: public polygon
{
public:
```

```

void display( ){
cout<<"The length of the triangle="<<length<<endl;
cout<<"The height of the triangle="<<height<<endl;
}
void area( )
{
cout<<"The area of the triangle="<<0.5*length*height<<endl;
}
};
int main( )
{
polygon *pg;
rectangle rec;
rec.input( );
rec.display( );
pg=&rec;
pg->area( );
triangle tg;
tg.input( );
tg.display( );
pg=&tg;
pg->area( );
return 0;
}

```

Output:

Question: 6

WAP in C++ to show the concept of virtual base class.

```

#include<iostream>
using namespace std;
class Base
{
    protected:
        int w;

```

```

        public:
            void get_w( )
            {
                cout<<"Enter the value to w of a Base class:"<<endl;
                cin>>w;
            }
void display_w( )
    {
        cout<<"The value to w of a Base class="<<w<<endl;
    }
};

class derived1: virtual public Base
{
    protected:
        int x;
    public:
        void get_x( )
        {
            cout<<"Enter the value to the x of derived1:"<<endl;
            cin>>x;
        }

void display_x( )
{
cout<<"The value to the x of a derived1="<<x<<endl;
}
};

class derived2:public virtual Base
{
protected:
int y; public:
void get_y( )

```

```

{
cout<<"Enter the value to the y of a derived2:"<<endl;
cin>>y;
}

void display_y( )
{
cout<<"The value to the y of a derived2="<<y<<endl;
}
};

class derived: public derived1,public derived2
{
protected:
int z;
public:
void get_z( )
{
cout<<"Enter the value to the z of a derived class:"<<endl;
cin>>z;
}
void display_z( )
{
cout<<"The value to the z of a derived class="<<z<<endl;
}
void display( )
{
cout<<"The sum is:"<<w+x+y+z<<endl
}
};

int main( )
{
derived d;

```



```
d.get_w( );  
d.get_x( );  
d.get_y( );  
d.get_z( );  
d.display_w( );  
d.display_x( );  
d.display_y( );  
d.display_z( );  
d.display( );  
return 0;  
}
```

Output:

Conclusion:

- What did you learn from the lab?